

## Analysis: Dire Straights

Dire Straights had one of the easiest small datasets for round 3. Several brute force or backtracking solutions that followed the rules could come up with a valid answer in the allotted time, so rather than look at the small dataset, we will instead examine the large. The large dataset requires some insight into a greedy approach.

For a problem like this, a good strategy is to think of how you might try to solve this problem by hand. A very intuitive strategy is to first put all the cards in order, then start setting the cards down on the table, creating a new straight whenever necessary. Since our goal is to make the length of the shortest straight as long as possible, then one idea that seems like it might work is to always increase the length of the shortest straight when we have a choice. Now we need only to prove that this choice is optimal.

Suppose we have two straights, one from **a** to **b**, another from **c** to **d**, such that  $a < c \leq d < b$ .

```
a-----b
c-----d
```

Notice that we can replace these two straights with straights from **a** to **d** and **c** to **b** (this change is illustrated below), and this does not decrease the score. In fact, this change has the potential to increase the score.

```
a-----d
c-----b
```

This shows that we can always make sure that a straight that started later never ends before one that started earlier. Hence, attaching the next card to the shortest straight is optimal.

Finally, the size of every straight is examined and the length of the shortest straight is the total score achieved in Dire Straights.

## Analysis: Irregular Cakes

Before we can start solving the problem, we first need to know how to calculate the area of the cake, so we know how much cake each party-goer is going to eat. We are guaranteed that this is a non-intersecting polygon in the limits, so we can use the equation:

$$2 * \text{Area} = \sum (X_i * Y_{i+1} - X_{i+1} * Y_i) \text{ for } i = 0 \text{ to } N-1.$$

For this to work the first and last points in the polygon must be the same point. That is,  $X_0 = X_N$  and  $Y_0 = Y_N$ . The list of points must also be in counterclockwise order. This can be achieved by iterating over all points in L, and then iterating over all points in U in reverse order.

Once we know the total area of the cake, we need to determine how much cake to give each guest. Since there are G guests, we can evenly calculate this number as  $\text{AreaPerGuest} = \text{Area} / G$ .

Finally we are ready to determine where to cut the cake. This can be solved a variety of ways, but here we will discuss using a binary search as this is often the simplest algorithm to code. To obtain a piece of cake with an area of AreaPerGuest, we know that the vertical cut will have an X coordinate between 0 and W, so we do a binary search over this range. For each cut point we try during the search, we compute the area of the cake to the left of that cut. If it produces a piece of cake with area greater than AreaPerGuest, we update the upper bound of our search. If we choose a cut that produces a piece of cake with area less than or equal to AreaPerGuest, we update the lower bound of our search. This search will eventually converge to the correct cut point with sufficient accuracy.

If  $G=2$ , then we are done. If  $G > 2$ , then the binary search can be repeated to find the other cuts. The second cut point should be placed such that the area of cake to the left of that cut is  $\text{AreaPerGuest} * 2$ , the third cut point should be placed so that the area of cake to the left of that cut is  $\text{AreaPerGuest} * 3$ , and so on.

At each iteration of the binary search, we need to calculate the area of a polygon using part of L, U, and a line on an arbitrary X coordinate, which requires more work than computing the area of the whole cake. The first thing to note is that we can use all points in L and U that have X coordinates such that  $0 \leq X_i \leq X_{\text{cut}}$ , where  $X_i$  is the X coordinate of point i, and  $X_{\text{cut}}$  is the X coordinate of the cut. Next we need to determine the Y coordinates of our  $X_{\text{cut}}$  points on L and U. If we find two consecutive points A and B in L such that  $A_x \leq X_{\text{cut}}$  and  $B_x > X_{\text{cut}}$ , we can use line intersection to find the intersection of line AB and the vertical line defined by  $X_{\text{cut}}$ . Once we know these two new points the area of the cut piece of cake can be calculated.

This problem can also be solved with a linear time solution in  $O(N+G)$ . The basic premise is to split the cake into trapezoids, and iterate from left to right accumulating the total area. Any time a trapezoid needs to be split for a cut, a quadratic equation is used to determine where to cut the cake. This approach only requires the basic formula for the area of a trapezoid, which is  $(a+b)/2 * h$ , where a and b are the lengths of the bases, and h is the height of the trapezoid.

### More information:

[Polygon area](#)

[Binary search](#)

# Analysis: Mystery Square

## Overview

This problem may have a simple statement, but make no mistake: it is really hard. Unless you have a small army of computers working in parallel, there is no way to try all  $2^{40}$  possible values. You need a way to limit your search.

The key observation is that for the most part, a perfect square is uniquely determined by *both* the first half of its digits, *and* by the second half. Either one suffices. So the high-level idea is to pick the half that has fewer question marks in it, try all possible ways of filling the question marks, deduce what the rest of the number has to be, and then see if it works. Before we get into the details though, let's talk about one nasty little implementation detail that you can't help but notice.

## Dealing with big integers

If a binary number has 120 digits, there is obviously no way to fit it into a standard 64-bit integer! And that means arithmetic can be a nuisance. Here are a few ways you can deal with this extra complication:

- Use Java and take advantage of the BigInteger class.
- In g++, you can use the little known `__uint128_t` type.
- Roll your own BigInteger functions for 128 bits. In practice you only need Square and SquareRoot. The first is doable with grade-school long multiplication formulas and some care. The second can be done with a binary search.
- Use an external library such as GMP.

Hopefully you remembered your Fair Warning from last year, and were prepared! Even so, we would have loved to let you work on only 64-bit integers if we could, but it turns out computers are so fast today that you can simply loop over ALL 64-bit perfect squares in a few seconds. The problem becomes pretty boring in that case.

## Filling in a perfect square top-down

All right, so with that detail of the way, let's get down to the solution. If you know the first half of the digits in a perfect square, how can you easily figure out the rest? For example, let's look at 10110???? (in binary).

Notice the square root has to be at least  $\text{sqrt}(1011000000)$  and at most  $\text{sqrt}(1011011111)$ . In fact, there is only one integer that is between these two, and it is 11011! So we can just see if  $11011^2$  matches 10110????, and then we're done. And this always works. If a number  $X$  has  $2t$  digits, then its square root  $Y$  has  $t$  digits, and  $(Y+1)^2 = Y^2 + 2Y + 1$ , which already differs from  $Y^2$  in more than the last  $t$  digits.

So in summary: once we know the first half of the digits in  $N$ , we can just replace the ? characters with 1, take the square root, round down, and that is the only possible option.

## Filling in a perfect square bottom-up

The other half of the solution is not much harder conceptually, but the devil is in the details. If you know the second half of the digits in a perfect square, how can you easily figure out the rest? For example, let's look at ???011001.

Getting started is actually tricky, but let's suppose we have figured out the last two binary digits of the square root are 01.

- The square root must then be  $4A + 1$  for some integer  $A$ . Its square is  $16A^2 + 8A + 1 = 8A + 1 \pmod{16}$ . However, we know that the square is  $9 \pmod{16}$ , and hence  $A$  must be odd. Therefore, the square root must end in 101.
- We now know the square root must be  $8B + 5$  for some integer  $B$ . Its square is then  $64B^2 + 80B + 25 = 16B + 25 \pmod{32}$ . However, we know that the square is  $25 \pmod{32}$ , and hence  $B$  must be even. Therefore, the square root must end in 0101.
- Continuing in this way, we can use the last  $k+1$  digits of  $N$  to calculate the last  $k$  digits of its square root. If you know just over half of the digits in  $N$ , this is enough to completely determine the square root. As above, we can now just check if it works, and then we're done.

This technique always works, subject to two conditions: (a)  $N$  must be odd, and (b) you must already know the last two digits of the square root. You might enjoy writing down the formula in both failure cases, and seeing what goes wrong.

Let's think about (b) first. If  $N$  is odd, then the square root must also be odd, and so the last digit must be 1. There is no easy way to determine what the second last digit has to be in advance, but who cares? Just try both of them, and see which one works!

Next let's suppose  $N$  is even. Since it is a perfect square, it must actually be a multiple of 4, and  $N/4$  is also a perfect square. So we can just cut the last two digits off of  $N$ , repeat until  $N$  becomes odd, and then solve as above. In fact, this trick is pretty much required. If  $N$  is odd, then the last  $k$  digits are enough to find  $k-1$  digits in the square root. If  $N$  is even though, then the last  $k$  digits may only be enough to find  $k/2$  digits in the square root.

Of course, we might not know whether  $N$  is even or odd in advance. If the last digit is a '?' character, then we just try both possibilities and see what happens.

*Remark:* This whole approach works only because 2 is prime. If we were working base-10, then an even number that is not a multiple of 10 would be pretty nasty to deal with!

## Putting it all together

Here is the full solution:

- Assume that  $N$  is odd, if possible.
- If  $N$  has more question marks in its bottom half than in its top half, then iterate over all possible ways of filling in the top half question marks, deduce the whole number, and see if it works.
- If  $N$  has more question marks in its top half than in its bottom half, then iterate over all possible ways of filling in the bottom half question marks, deduce the whole number, and see if it works.
- Now assume that  $N$  is even, if possible. Fill in the last two digits as zero, and repeat from the very beginning (potentially solving either top-down or bottom-up) for  $N/4$ .

The parenthetical comment in the last part is actually quite important! Consider the following input for example:

"10010000010000011100000110110010001????????????????????????????????????  
000000000?000000000000000?0000000000000000000000000?00". Most of the '?' characters are in the first half, so it is tempting to just start from the back and never re-evaluate your decision. However, there are only '0' characters back there, and they will not give you much information. To run fast enough in this case, you need to start from the front after eliminating some of the 0's.

## Analysis: Perpetual Motion

The first thing we need to notice is that if two lemmings do not end up in the same square after one second, they will never end up in the same square. That is because after one second there will be exactly one lemming in each square, and the state is exactly the same as the second before.

The number of combinations for the conveyor belt directions is  $2^{R \cdot C}$ , so we can afford to try all of them to solve the easy input and count how many combinations lead to all lemmings being in different squares after one second.

To solve the hard input, we'll have to take a look at the problem from a graph theory perspective. Suppose we created a bipartite graph like this:

- For each cell  $(r, c)$ , create two nodes  $\text{start}_{r, c}$  and  $\text{end}_{r, c}$ .
- Create an edge between  $\text{start}_{r_1, c_1}$  to  $\text{end}_{r_2, c_2}$  if there is a way to choose the direction of the conveyor belt in cell  $(r_1, c_1)$  such that the lemming ends up in  $(r_2, c_2)$  after one second.

All start nodes will end up being incident to exactly two edges because there are exactly two different cells where a lemming can end up in one second starting from any given cell. End nodes, on the other hand, can be incident to 0-8 edges, depending on the conveyor belt orientations of the neighbouring cells.

Let's observe the graph some more. There are two rules that apply:

1. If there are no edges incident to a node  $\text{end}_{r, c}$  then no lemming can end up in cell  $(r, c)$  in one second, so there will be two lemmings in some other cell no matter how we direct the conveyor belts. In that case the answer is simply 0, so we can proceed to the next test case.
2. If there is a node  $\text{end}_{r_2, c_2}$  incident to only one edge leading to  $\text{start}_{r_1, c_1}$  then we have no other choice but to direct the conveyor belt on the cell  $(r_1, c_1)$  to lead to the cell  $(r_2, c_2)$ . Then we can simply remove both  $\text{end}_{r_2, c_2}$  and  $\text{start}_{r_1, c_1}$  from the graph along with all the incident edges.

We can apply these rules iteratively until they can not be applied anymore.

Let  $N$  be the number of start nodes left after the above process is done. The number of end nodes is also equal to  $N$ , because we removed them in pairs.

The number of edges incident to each start node is still equal to two, so total the number of edges is equal to  $2 \cdot N$  because the graph is bipartite. We also know that each end node is now incident to at least two edges, because the above rules do not apply anymore.

But, if any of the end nodes had more than two incident edges then the number of edges incident to all the end nodes combined would be greater than  $2 \cdot N$ . This would contradict the fact that the number of edges is equal to  $2 \cdot N$ . Therefore, the number of edges incident to each end node is also equal to two.

Any graph having all node degrees equal to two is in fact a set of cycles. For bipartite graphs, the length of each cycle is even. So, we are left with  $K$  cycles of even length which we can solve independently and multiply the individual numbers to get the final number.

To solve the circle, select any cell  $(r_1, c_1)$  and pick a conveyor belt direction. The lemming ends up in  $(r_2, c_2)$ . Now remove  $\text{start}_{r_1, c_1}$  and  $\text{end}_{r_2, c_2}$  from the graph along with incident edges. This will break up the cycle, and we can proceed to apply rule #2 until we decide the conveyor belt for all the remaining cells. Because the cycle has even length and we always remove nodes in pairs, the rule #1 will never apply.

We can pick the direction of the cell  $(r_1, c_1)$  in two ways. So the answer for any cycle is always 2. So the final answer is equal to  $2^K$  (modulo 1000003).